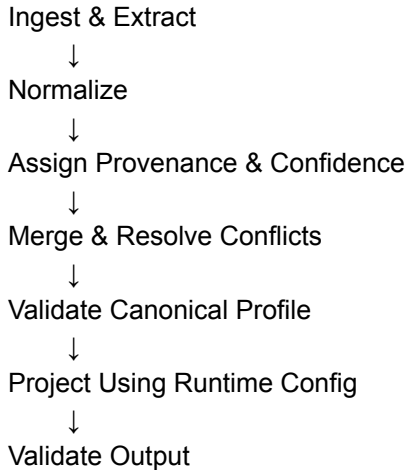


# Candidate Data Transformer–Technical Design

**Author:** Nikita Gupta

**Role:** Backend/Data Engineer Intern Assignment

## 1. Pipeline Architecture



## 2. Canonical Profile Schema

Candidate

- |— candidate\_id
- |— full\_name
- |— emails[]
- |— phones[]
- |— location
- |— skills[]
- |— experience[]
- |— education[]
- |— provenance[]
- |— overall\_confidence

A deterministic candidate ID is generated from normalized identity fields to ensure repeatable output.

## 3. Merge & Conflict Resolution

### Identity Matching

Merge records sharing a normalized email/phone number.

### Conflict Resolution

1. Highest confidence wins.
2. If tied: Recruiter CSV > Resume.
3. If still tied: lexicographically first value.

## Confidence

- Recruiter CSV: **0.95**
- Resume Parser: **0.75**
- Multi-source agreement increases confidence (capped at 0.99).

## 4. Runtime Projection Layer

Runtime JSON configuration reshapes the canonical profile into consumer-specific output without changing the core pipeline.

Supported features:

- Select output fields
- Rename fields
- Field mapping
- Toggle provenance/confidence
- Missing value handling (null, omit, error)

## 5. Edge Cases

- Malformed CSV rows skipped with warnings.
- Invalid emails/phones removed.
- Duplicate skills merged after normalization.
- Missing sources handled gracefully.
- Unknown values returned as null.

## 6. Scalability

Normalization and merging operate in **O(N)** time, while identity matching uses hash-based lookups for efficient processing of large datasets.

## 7. Out of Scope

- Fuzzy name matching
- OCR for scanned resumes
- ML/NLP-based extraction
- Cross-language skill normalization
- LinkedIn scraping